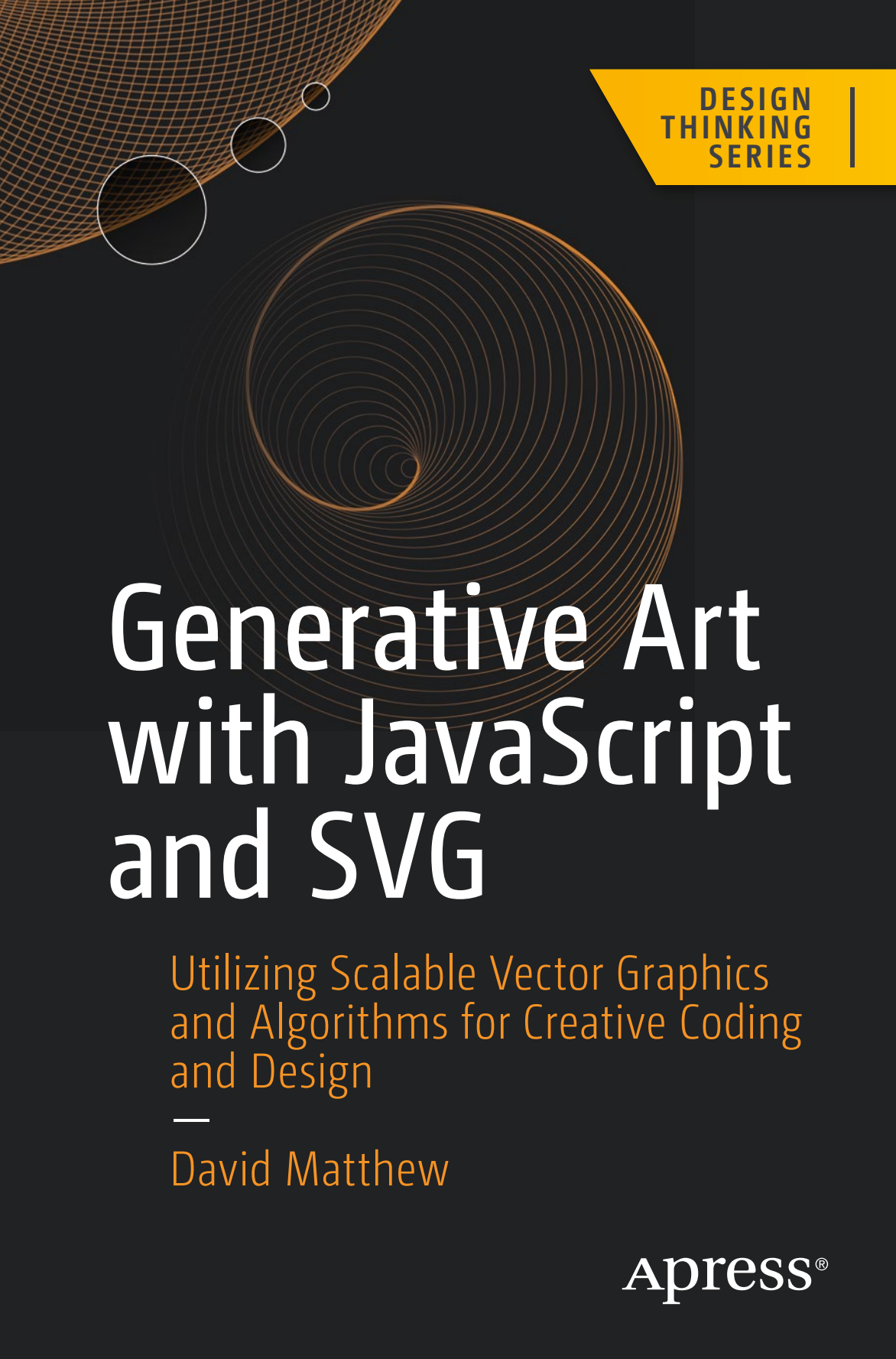




DESIGN
THINKING
SERIES

Generative Art with JavaScript and SVG

Utilizing Scalable Vector Graphics
and Algorithms for Creative Coding
and Design

David Matthew

Apress®

Design Thinking

Design Thinking is a set of strategic and creative processes and principles used in the planning and creation of products and solutions to human-centered design problems.

With design and innovation being two key driving principles, this series focuses on, but is not limited to, the following areas and topics:

- User Interface (UI) and User Experience (UX) Design
- Psychology of Design
- Human-Computer Interaction (HCI)
- Ergonomic Design
- Product Development and Management
- Virtual and Mixed Reality (VR/XR)
- User-Centered Built Environments and Smart Homes
- Accessibility, Sustainability and Environmental Design
- Learning and Instructional Design
- Strategy and best practices

This series publishes books aimed at designers, developers, storytellers and problem-solvers in industry to help them understand current developments and best practices at the cutting edge of creativity, to invent new paradigms and solutions, and challenge Creatives to push boundaries to design bigger and better than before.

More information about this series at <https://link.springer.com/bookseries/15933>.

Generative Art with JavaScript and SVG

Utilizing Scalable Vector
Graphics and Algorithms for
Creative Coding and Design

David Matthew

Apress®

Generative Art with JavaScript and SVG: Utilizing Scalable Vector Graphics and Algorithms for Creative Coding and Design

David Matthew
Limerick, Ireland

ISBN-13 (pbk): 979-8-8688-0085-6

<https://doi.org/10.1007/979-8-8688-0086-3>

ISBN-13 (electronic): 979-8-8688-0086-3

Copyright © 2024 by David Matthew

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

Trademarked names, logos, and images may appear in this book. Rather than use a trademark symbol with every occurrence of a trademarked name, logo, or image we use the names, logos, and images only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The use in this publication of trade names, trademarks, service marks, and similar terms, even if they are not identified as such, is not to be taken as an expression of opinion as to whether or not they are subject to proprietary rights.

While the advice and information in this book are believed to be true and accurate at the date of publication, neither the authors nor the editors nor the publisher can accept any legal responsibility for any errors or omissions that may be made. The publisher makes no warranty, express or implied, with respect to the material contained herein.

Managing Director, Apress Media LLC: Welmoed Spahr

Acquisitions Editor: James Robinson-Prior

Development Editor: Jim Markham

Editorial Assistant: Gryffin Winkler

Cover designed by Isaac Soler at eStudioCalamar and David Matthew

Distributed to the book trade worldwide by Springer Science+Business Media New York, 1 New York Plaza, 1 FDR Dr, New York, NY 10004. Phone 1-800-SPRINGER, fax (201) 348-4505, e-mail orders-ny@springer-sbm.com, or visit www.springeronline.com. Apress Media, LLC is a California LLC and the sole member (owner) is Springer Science + Business Media Finance Inc (SSBM Finance Inc). SSBM Finance Inc is a **Delaware** corporation.

For information on translations, please e-mail booktranslations@springernature.com; for reprint, paperback, or audio rights, please e-mail bookpermissions@springernature.com.

Apress titles may be purchased in bulk for academic, corporate, or promotional use. eBook versions and licenses are also available for most titles. For more information, reference our Print and eBook Bulk Sales web page at <http://www.apress.com/bulk-sales>.

Any source code or other supplementary material referenced by the author in this book is available to readers on the Github repository. For more detailed information, please visit <https://www.apress.com/gp/services/source-code>.

Paper in this product is recyclable

Dedicated to the deeply human experience of creating art.

Table of Contents

About the Author	xv
About the Technical Reviewer	xvii
Acknowledgments	xix
Introduction	xxi
Chapter 1: The Beginner's Path	1
Why JavaScript and SvJs?	1
Introducing Scalable Vector Graphics	2
Native SVG	3
Generating SVG	5
Getting Set Up	8
The Code Editor	8
Node.js and NPM	8
Initializing and Installing SvJs	9
Scaffolding Our Sketches	10
Serving Our Sketches	12
Our First Generative Sketch	13
Summary	16
Chapter 2: A Programming Primer	19
Syntax	19
Case Sensitivity	20
Spacing	20

TABLE OF CONTENTS

Semicolons	20
Comments	21
Values	22
Numbers	22
Strings	22
Booleans	24
Empty Values	24
Variables	25
Operators	26
Arithmetic Operators	26
Comparison Operators	29
Logical Operators	30
Conditional Statements	31
Loops	33
The While Loop	34
The For Loop	35
Functions	37
Function Parameters	37
Invoking Functions	38
Scope	40
Anonymous Functions	42
Arrow Functions	42
Objects and Arrays	44
Arrays	44
Objects	47
Classes	49
Idiosyncrasies and Other Features	50
Null and NaN Weirdness	51

Secret Casting	51
Semicolon Uncertainty	52
Summary.....	53
Chapter 3: All About SVG	55
The Parent SVG Element	55
The Viewport and ViewBox.....	57
Setting and Getting Values.....	59
Quicker Element Creation	60
Lines and Shapes.....	61
Rectangles and Squares.....	61
First Strokes	63
Circles and Ellipses	66
Lines, Polylines, and Polygons.....	68
Text and Titles	72
Definitions	75
Gradients.....	76
Patterns.....	78
Grouping and Reusing Elements.....	83
Summary.....	84
Chapter 4: Randomness and Regularity	85
Analogue and Digital Randomness	85
The SvJs Gen.random() Function	87
Elements Everywhere All at Once	88
Varying Color and Opacity	89
Varying Element Selection.....	91

TABLE OF CONTENTS

Regular Grids	94
The Nested For Loop.....	94
A More Flexible Grid	96
Clip Paths and Color Palettes.....	98
Arrays of Colors	98
Clipping Our Content.....	100
Choice and Chance	105
The SvJs Gen.chance() Function.....	105
Chance in Action.....	106
Probability Distributions.....	108
Uniform Distribution	108
Gaussian Distribution	109
Pareto Distribution.....	113
Masking Our Content.....	115
Summary.....	118
Chapter 5: The Need for Noise	119
Random Limits.....	119
Making Noise	119
Noise Explained	120
The SvJs Noise Module	122
Into the Noise Matrix.....	124
A Noisy Grid	124
Mapping the Noise Values	128
Optimize with Style.....	129
Spinning Noise.....	130
Mapping and Constraining.....	132
Rotating and Translating.....	133
Summary.....	134

Chapter 6: The All-Powerful Path	137
The Path Element.....	137
D for Data	138
Path Commands	138
Starting and Ending a Path	139
Straight Lines.....	140
The Simple L.....	140
Horizontal and Vertical Varieties.....	142
Further Economies	143
Quadratic Bezier Curves	144
Control Points	144
A Smooth Shortcut	145
A Quadratic Slinky	147
Elliptical Arcs	150
Setting the Flags	151
Irregular Radii.....	152
Generative Arcs	153
Cubic Bezier Curves.....	156
Cubic Control Points	156
S for Symmetry.....	158
Organic Curves	159
An Easier Option	163
Summary.....	165
Chapter 7: Motion and Interactivity.....	167
Event Listeners	167
Event Types.....	168
Event Parameters	169
Triggering the SvJs Save Method.....	170

TABLE OF CONTENTS

Creative Cursor Tracking	171
The SvJs trackCursor() Method.....	171
Interactive Ellipses	172
Programming Motion	174
CSS Keyframes	177
The SMIL Way	179
The Web Animations API	181
The requestAnimationFrame() Method	184
Methods Summarized.....	188
Collision Detection	190
Setting Boundaries	190
Initializing and Extending Our Shapes.....	191
Frame-by-Frame Calculations	193
Circularity.....	196
Slices of PI.....	198
Sine and Cosine.....	200
Animating Our Circles.....	200
Summary.....	202
Chapter 8: Filter Effects	205
Filter Fundamentals	205
The Ins and Outs.....	207
The SvJs createFilter() Method	209
The Filter Region	210
Filter Effects 101	212
Shadows.....	212
Coloring	213
Blending	216

TABLE OF CONTENTS

Compositing.....	217
Noise and Distortion.....	221
Turbulence	221
Displacement.....	223
Creating a Cosmic Bubble	224
Lighting and Texture.....	228
Diffuse and Specular Lighting	230
Light Sources.....	231
Simulating Textures	232
Generative Textures	235
Summary.....	240
Chapter 9: The Generative Way	243
The Journey So Far	243
The Voyage Forward.....	244
Trigonometry	244
Fractals.....	246
Systems Simulations	249
Closing Comments	251
Index.....	253

About the Author



David Matthew is a web developer, designer and musician who is passionate about the intersection of art, music, and computer science.

He writes about creative coding, generative art, music visualization and other related themes on his website davidmatthew.ie. He is also a regular producer of generative art and shares his work on Instagram (@davidmatthew_ie).

About the Technical Reviewer



Eoin Ó. Conchúir, PhD, is the founder of Bitesize Irish for Irish language learners. His art can be found on vibrantconflux.com.

Acknowledgments

I want to thank first those who should always come first: my incredible wife Sandra, my beautiful boy Noah, my constant companion Kow, and my amazing mum, who has always believed in me.

Thank you to my technical reviewer Eoin Ó Conchúir, whose expertise and rigorous code checking weeded out many an error and prompted further clarity on points I had left overly vague. Your contribution means a better book overall, and any remaining errors are mine alone.

Thank you to my editing team at Apress: James Robinson Prior, Gryffin Winkler, and James Markham. In the 15 months since embarking on this project, I've moved house, changed jobs, and, most life-changing of all, welcomed a newborn into my life. Amidst all this change (and sometimes chaos), you've remained exceptionally understanding, flexible, and enthusiastic.

In the broader generative art and creative coding community, I'd like to thank some notable individuals who have inspired my own work and continue to amaze me with their generosity and ingenuity: Pablo Almunia, whose excellent gySVG library was the inspiration behind SvJs and who provided invaluable insights for the first chapter; Ahmad Moussa (a.k.a. Gorilla Sun) whose writing keeps so many of us artists in the loop; Sara Soueidan, whose pioneering work on SVG filters I always find myself returning to; Matt Pearson, one of the first authors to ignite in me that generative spark; Keith Peters, for helping me and many other coders be less afraid of math; and Dan Shiffman, the genius behind the classic work *The Nature of Code*, and whose enthusiasm is so infectious I can't quite capture it in words (go watch his YouTube videos under the moniker *The Coding Train* to see what I mean).

ACKNOWLEDGMENTS

Finally, thank you to my friends and family for all the support. Daire, for your sense of humour (and pretending to NFT anything I post to Instagram); Stephen (founder of letsfundit.ie, a charity well worth a plug) for the office craic and always asking how the book was going. Steven S, for getting me through tough times. To my new team at UL, thank you for being so welcoming and all-round amazing. And to my siblings Brian, Paul, Claire and Sarah, thanks for promising to read this book cover to cover (just joking - but you need to at least read the back!).

Introduction

A caveat before we begin: generative art, and creative coding more generally, can become an all-consuming addiction. “Just one more iteration” is a refrain you might hear from those unfortunate artists caught in its thrall. Creatives craving that next generative hit mixed with just the right amount of randomness.

The purpose of this book is to deal out this computational devilry in small doses, in a clear, logical fashion, so you can approach this topic safely and with your sanity intact. We will encounter organic forms borne of mechanical precision, unpredictable swathes of color cover our screens, lines and shapes flocking together in impossible harmony, and see complex structures emerge from simple rules. This is the essence of generative art, where not all is in your control.

That said, it’s a medium that requires time, patience, and skill on the part of the artist – no less so than with other artistic mediums. And although I might be exaggerating the power it has over its practitioners, the point I really want to make is that coding has this expressive side to it – it can be frustrating and headache-inducing, sure – but it can also fascinate and inspire. If I can equip you with some new generative tools and techniques, and in the process excite you just a little about their creative possibilities – possibilities that extend beyond the field of art into those of design and web development – this book will have more than done its job.

What You Should Know

As a reader, what should you know before tackling this book? It will no doubt be easier if you are comfortable with the core concepts of programming, like variables, functions, loops, and conditionals. These kinds of concepts carry over from language to language, differing mainly with respect to syntax, so if you come from a language other than JavaScript, then that's completely fine.

If you have no prior programming experience, we will be covering the concepts required in Chapter 2, but I should emphasize that this book is not intended to be an introduction to programming. The second chapter is best thought of as a primer, a jumping-off point for a deeper dive elsewhere. A short introductory course would really benefit you here. Personally I'd recommend the wonderful freeCodeCamp.com, a site I've spent many hours using myself. It offers free interactive tutorials on the basics of HTML, CSS, and JavaScript, and much more besides.

For those coming from another language, like Python, PHP, or C#, the second chapter may still be valuable, to be sure you're acquainted with the syntax and some of the idiosyncrasies of JavaScript.

If you're already a practicing web developer, you could skip (or quickly skim) Chapter 2, depending on your level of experience. If you're unfamiliar with any of the techniques or syntax used in our opening example (which we'll end Chapter 1 with), I'd encourage you to give the second chapter a read, where they'll be fully explained. The field of web development is vast enough that what is bread and butter to one developer might be bleeding edge to another.

What You Will Learn

I mentioned previously that I'll be equipping you with some new tools and techniques. Let's flesh these out.

Tools

On the tool end of things, we'll be using the following:

- JavaScript and the SvJs library. I'll elaborate on this further later.
- A code editor. Here I'll be giving you a choice between
 - Visual Studio Code (an excellent open source code editor)
 - CodePen (a popular online code editor)
- If you opt to use Visual Studio Code (which I recommend), we'll also be using Node.js and NPM to manage dependencies (a good habit to get into as a developer).

All code examples are available on both GitHub and CodePen, so feel free to follow along whichever way you prefer. You can of course do both, work locally with a code editor but check out CodePen examples online for quick inspiration. For convenience, I'll organize and embed all CodePen examples at davidmatthew.ie/generative-art-javascript-svg.

Techniques

In terms of the techniques we'll tackle, by the end of this book, you should have a good overview of the following:

- The main functionality of the SvJs library and how it relates to the SVG spec
- Creating generative art sketches with JavaScript using ES6+ syntax
- Generating primitive SVG shapes

INTRODUCTION

- Creating iterative variations of sketches by randomizing parameters
- Using noise to create organic variance
- Generating complex SVG paths
- Making sketches interactive
- Animating sketches
- Using SVG filters generatively

We'll be covering a lot of ground, so don't put yourself under pressure to understand everything. I must have encountered certain programming concepts umpteen times before the proverbial penny finally dropped (JavaScript promises anyone?), and whenever understanding did finally dawn, it would usually be because I was experimenting with something I *wanted* to build, rather than repeating tutorial steps by rote. Tutorials and instructional books certainly have their place (I wouldn't be writing this book otherwise), but it's important that you take what *you* want from them, rather than seeing them as syllabi to be strictly followed.

A sense of play is important, particularly when it comes to art. And although I mentioned tools previously, I would prefer if you viewed this book as more of a toybox than a toolbox. When I think of tools, I think of problems that need fixing, like the loose hinges on that crooked cabinet you're going to tighten any day now. Tools tend to be more functional than fun, and I'd like you to have some fun with this book.

CHAPTER 1

The Beginner's Path

Before journeying along any path, the groundwork needs to be in place. In this opening chapter, that's what we'll do, lay the groundwork. We'll introduce SVG, explain what makes it a uniquely powerful image format, and show how it can be used with JavaScript to create generative art. In the process, we'll set up our tools and a template we can use for subsequent sketches.

Why JavaScript and SvJs?

Most books about generative art use a Java-based language called Processing, or its JavaScript port p5.js. Processing was created specifically for artists and designers new to coding and has a large and active community. So why doesn't this book use it?

My first forays into generative art were with Processing, so I certainly acknowledge its value. I quickly moved to p5.js when the library was first released in 2013, which allowed generative sketches to be written directly in JavaScript, the language of the Web. But when I wanted to integrate some of my own sketches into real-world web development projects, its limitations quickly showed. It's a *large* library, clocking in at close to a megabyte at last check, and while that may not sound like much, it's a lot by web development standards.

The p5.js library is built on top of the HTML Canvas API, which I soon discovered is actually quite straightforward to use. Using this API directly, I was able to achieve much the same results as with p5.js, so that became my go-to. However, the output of all my sketches – p5.js or Canvas – was still resolution-dependent bitmap graphics, devoid of any semantic content. What does that mean, and why does it matter (to me at least)? Let me explain.

Introducing Scalable Vector Graphics

Back in the early days of the Web, when dial-up modems were dominant and connecting to the Internet was anything but instant, bandwidth came at a premium. File sizes had to be super small if you wanted a page to load in your lifetime, and bitmap images, such as JPGs and PNGs, were the main bandwidth bottleneck.

Bitmap images – also known as raster images – are comprised of large chunks of data (or bits), and generally speaking, if you want a larger image, you need more bits, which means a bigger file size. SVG, on the other hand, is a vector format, which is fundamentally different. SVG can be scaled to any dimension, all without a corresponding increase in file size, and it is always dazzlingly sharp and crisp. This is possible because it doesn't bother itself with the bits (i.e., pixels) needed to paint the image to the screen, but rather describes the image to be rendered at a more abstract, semantic level. And it does this in much the same way that HTML describes the structure and content of a web page. As the Mozilla Developer Network puts it

SVG is, essentially, to graphics what HTML is to text.

The SVG format was not only a powerful solution to a practical bandwidth problem; it was a nonproprietary format officially standardized by the World Wide Web Consortium (which is as good as future-proofing gets in the world of web technology). The stage was set for its adoption as far back as 1999, but unfortunately browser vendors dragged their feet and were extremely slow to introduce support. So for the longest time (in Internet terms at least), SVG languished in obscurity. It only really got the support it deserved with the decline of Flash (Adobe's proprietary format that required a plug-in to run and was often riddled with security vulnerabilities) and the rise of responsive design and retina (or high PPI) screens, where its scalability and sharpness really shine.

These days we can use the format freely without worry, but to use it for the odd icon and logo is one thing; to tap its full potential is another.

Native SVG

As SVG is a declarative language like HTML, it's very human-readable and easy to get started with. Just like HTML elements, SVG elements are written using opening and closing angle brackets and contain attributes with values. Here, for example, is how you'd create a circle:

```
<circle r="125" cy="250" cx="250" fill="cyan"/>
```

The attributes `r`, `cx`, `cy`, and `fill` in the preceding example refer to the circle's radius, the `x` and `y` coordinates of its center, and the color to fill it with. All are sensibly named and simple to follow.

Some SVG elements will contain other elements nested within them, referred to as their child nodes or children, and will therefore need opening and closing tags. One prominent example is the parent `<svg>` element itself, which contains all other SVG elements.

As an example of how you might handwrite an SVG, here is the markup underlying a very simple composition in the style of Hilma af Klint, arguably the first abstract artist in Western art history.


```
<svg width="500" height="500" style="background-color:
#ad3622">
  <title>A simple Hilma af Klint-inspired knock-off</title>
  <circle r="125" cy="250" cx="250" fill="#d0d1c9"/>
  <circle r="100" cy="250" cx="250" fill="#1c1c1c"/>
  <circle r="75" cy="250" cx="250" fill="#5085b4"/>
  <circle r="50" cy="250" cx="250" fill="#d6a946"/>
</svg>
```

As you can see, SVG is written in such a way as to preserve the semantics of the code. Search engines love this; no longer are they looking at an impenetrable wall of pixels; they can clearly see the *intent* within the markup. In this case, four circles displayed in the order they're written, with a title for extra accessibility. Figure 1-1 shows how this markup appears when rendered.



Figure 1-1. A simple composition in the style of Hilma af Klint's *Svanen (The Swan)*

Declarative formats are straightforward to read, but their weakness is that they can become tedious to write. What if we wanted 100 circles instead of four? With native SVG, we'd have to handwrite all of them, one after the other. And if we wanted to display our circles at random *cx* and *cy* positions, each with a randomly selected fill, this isn't possible at all. Variables, loops, functions, and all the fun stuff of imperative programming aren't available to us. Declarative formats are concerned more with the *what* of a program rather than the *how*. The *how*, however, will very much matter to us. Without the *how*, we wouldn't have algorithms, and algorithms are essential to generative art.

Algorithms are like recipes; they contain the steps you need to follow to achieve a certain result. If you're baking a cake, you don't just shout "Cake!" and expect one to materialize, no matter how specific you get with your declaration ("with strawberry icing and a caramel filling!"). This would be the declarative approach, and in practice, it sounds more than a little eccentric. To bake a cake, you follow a series of well-defined steps or instructions and have your ingredients and your oven at the ready. This is akin to the imperative approach in programming; it's more hands-on, and sometimes things get messy. But ultimately it gives you more creative control.

Generating SVG

So if native SVG doesn't allow for the use of algorithms, how do we write SVG using an imperative approach? The answer is we *script* our SVG, and this is where JavaScript and SvJs come into play.

We could script SVG directly in JavaScript without a library, but that too has its challenges, the main one being the verbose boilerplate code we'd have to write. The SvJs library saves us that trouble, making SVG more intuitive and fun to write. Here's an example of a simple SVG (Figure 1-2) written using vanilla JavaScript:

```

const svg = document.createElementNS('<http://www.w3.org/2000/
svg>', 'svg');
svg.setAttribute('width', '150px');
svg.setAttribute('height', '150px');
const div = document.getElementById('container'); div.
appendChild(svg);
const rect = document.createElementNS('<http://www.w3.org/2000/
svg>', 'rect');
rect.setAttribute('x', '0');
rect.setAttribute('y', '0');
rect.setAttribute('width', '150');
rect.setAttribute('height', '150');
rect.setAttribute('fill', 'cornflowerblue');
svg.appendChild(rect);

```

And the following snippet is the equivalent SVG written using SvJs (don't worry about the details just yet; all you need to note is how concise it is vs. vanilla JavaScript). The output of this code you can see in [Figure 1-2](#).

```

const div = document.getElementById('stage');
const svg = new SvJs().set({ width: '150px', height: '150px'
}).addTo(div);
svg.create('rect').set({
  x: 0, y: 0, width: 150, height: 150, fill: 'cornflowerblue'
});

```



Figure 1-2. *A cornflower blue-colored rectangle, in all its glory*

Now you *could* write some functions to make writing vanilla JavaScript less painful, but that leads you down the road of writing a whole host of other utility functions to make the basics less burdensome. With SvJs, I've done my best to save you this trouble. It's essentially a very thin wrapper over the real SVG spec with some helpful generative functions thrown in. This keeps its footprint extremely light while maintaining fidelity to the SVG spec.

SvJs was inspired by the gySVG library, a similarly light but more general-purpose JavaScript library that comes complete with a plug-in API and modern framework support. I was initially going to use gySVG and extend its functionality with a plug-in, but this plug-in soon grew to the point where it made more sense to write my own library with a specific focus on generative art. This was how SvJs came to be.

Getting Set Up

Enough with the preamble. Let's get ourselves set up to write some code.

The Code Editor

If you don't have VS Code already running on your machine, head on over to code.visualstudio.com and download and install the appropriate version for your operating system.

If you can't install a code editor on your machine or would just prefer to follow along online, you can use CodePen instead. Go to codepen.io and sign up an account if you're not already registered.

I will be writing the examples throughout this book with the VS Code editor and local files in mind; the same code can be found online via the CodePen versions with some minimal differences (such as how we import the SvJs library and the lack of HTML boilerplate which CodePen automatically provides).

Node.js and NPM

You can skip this part if you intend to use CodePen only, but I would still recommend getting familiar with Node.js if you have any plans on getting into the field of web development.

Node.js is an open source JavaScript runtime environment that allows you to set up your own JavaScript-powered server. It also allows you to manage dependencies via the included package manager NPM.

Go to nodejs.org and download and install the latest LTS (long-term support) version.

Initializing and Installing SvJs

Create a base folder where you'll save all the work related to this book, and name it something like `generative-svg`.

Open up VS Code, select **File** ➤ **Open File**, and navigate to this folder. Next, select the **Terminal** ➤ **New Terminal** command, and you should see a new window appear at the bottom of the screen referencing the current folder location. It should look something like this:

```
your-name@computer-name ~/Documents/generative-svg
```

The important thing is that the path references the folder you created in the last step (i.e., `generative-svg`). Once you've verified that, you're ready to initialize the project. To do this, type the following into the terminal:

```
npm init -y
```

What this does is run Node's package manager NPM, and the `init` command initializes the project, or "package." The `-y` flag just tells `npm` that we want to accept all default setup options. You should notice a new file has been created, called `package.json`. This is a file you'll see a lot of in web development, but you don't need to worry about its contents right now. Just know that its purpose is to manage our dependencies, the main one being the SvJs library.

To install SvJs, run the following command:

```
npm install svjs
```

This will add a new line in the `package.json` file, referencing the version of SvJs you've just installed. A `package-lock.json` file will also be created (a file you'll never need to look at) and a `node_modules` folder, where SvJs and any of its dependencies are stored (it doesn't have any so you won't see additional folders here).

Scaffolding Our Sketches

Our first sketch will require a little HTML and CSS before we tackle the JavaScript. We'll keep this markup and styling minimal and more or less identical throughout the book, as our real focus will be on the JavaScript.

Let's create a new folder called `sketches` inside the base folder, and inside `sketches` create a new folder called `00-template`. We'll number our folders so that they sort nicely. The `00-template` folder will contain the very basics we'll copy from one sketch to another. Inside `00-template` create a file called `index.html` and another called `sketch.js`. Once you've done that, you should have a folder structure like this:

```
generative-svg
|-- node_modules
|-- sketches
    |-- 00-template
        |-- index.html
        |-- sketch.js
    |-- package-lock.json
    |-- package.json
```

Open the `index.html` file. It's worth noting that VS Code can generate some boilerplate markup for you if you type `!` and press the tab key, but I'll include our boilerplate in full here so you can copy and paste it. Generally, I recommend you write out the example code yourself and limit the “control-c-control-v” activity where possible, as you're less likely to learn this way. Here, however, it's fine, as HTML and CSS aren't our focus.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
```

```

<meta name="viewport" content="width=device-width, initial-
scale=1.0">
<style>
  body {
    margin: 0;
    background-color: #202020;
  }
  #container {
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
  }
</style>
<title>SvJs Template</title>
</head>
<body>
  <div id="container"></div>
  <script src="./sketch.js" type="module"></script>
</body>
</html>

```

There's just a couple of things worth pointing out about the preceding code: our `<div id="container">` will be where the generative art actually happens. It will be our canvas, so to speak. I have placed it within the center of the screen, removed any default margins, and given the template an overall “dark-mode” feel (my own personal preference).

Below the `<div>` you'll see a `<script>` tag. This is where we'll pull in all our code, so any further modifications to the HTML will be unnecessary going forward (other than maybe updating the `<title>` tag for each sketch, but I'll leave that up to you). The script is set to `type="module"`. What this

does is allow us to handle the import of other files and libraries that are packaged (or exported) as modules. This is a good practice to get into, and it's also how we're going to import the SvJs library.

Open up the `sketch.js` file and include this as your first two lines:

```
// Import the SvJs library.  
import { SvJs } from '../..../node_modules/svjs/src/index.js';
```

Our template is now complete, ready to be copied for subsequent sketches. Well done!

Serving Our Sketches

If you navigate to the `00-template` folder at this point and double-click on `index.html` to open it in a web browser, you'll see some errors in the developer console (accessible by pressing F12 on most browsers). This is because we need to serve our HTML files using the `http://` protocol rather than the `file://` protocol (you'll see the protocol prefix if you check the full URL in the address bar).

To rectify this, we need to install an http server. And ideally one that detects file changes and instantly reloads our page, saving us from having to manually refresh our browser each time (which becomes a pain after a while). There's a neat little package called `live-server` that takes care of this for us. To install it, run the following in the terminal:

```
npm install live-server -g
```

The `-g` flag tells `npm` to install this package globally on our machine, rather than locally to the project in question. As it's more a general-purpose utility than a project-specific dependency, this is what we want.

To get `live-server` to run, all you need to do is type `live-server` into the terminal. It will then automatically open a new browser window where you'll see your project files (assuming you're still in the project's base

folder in VS Code). Navigate to sketches ► 00-template and you should see your page load free of any console errors. It's also free of any content though, so let's write some code to address this.

Our First Generative Sketch

As I mentioned previously, our first sketch will serve as a basis for explaining the fundamental programming concepts we'll be covering in Chapter 2, so what follows is, for this reason, rather light on explanations (other than some comments in the code itself).

Make a copy of the 00-template folder and call it 01-our-first-generative-sketch, ensuring to copy it to the same location (i.e., so that the parent folder is sketches).

Then, either write out the following code (recommended) or copy it into your sketch.js file, below the import statement. Read the comments as you go (the lines starting with //), which I've purposely kept quite verbose so you can get a better handle on what's happening.

```
// Import the SvJs library.
import { SvJs } from '../..../node_modules/svjs/src/index.js';

// Create some global variables.
const svgSize = window.innerWidth > window.innerHeight ?
window.innerHeight : window.innerWidth;
const bgColor = '#181818';

// Create an object to store some of our randomised parameters.
const randomised = {
  hue: random(0, 360),
  rotation: random(-180, 180),
  iterations: random(10, 100)
}
```

```
// Create our parent SVG and attach it to the element with id
'container'.
const svg = new SvJs(); svg.addTo(document.
getElementById('container'));

// Set the width and height of the viewBox and the displayed
size of the SVG.
svg.set({ viewBox: '0 0 1000 1000', width: svgSize, height:
svgSize });

// Create a background layer - a rectangle the full size of our
viewBox.
const rect = svg.create('rect');
rect.set({ x: 0, y: 0, width: 1000, height: 1000, fill:
bgColor });

// Run a loop a random number of times to create our ellipses.
for (let i = 0; i < randomised.iterations; i += 1) {

  // Set the centre point, the x and y radii of our ellipse and
  its rotation.
  let center = 500;
  let radiusX = 100 + (i * 3);
  let radiusY = 300 + (i * 2);
  let rotation = randomised.rotation + (i * 2);
  // If our random hue is less than 180, increment it.
  Otherwise decrement it.
  let hue;
  if (randomised.hue < 180) {
    hue = randomised.hue + (i * 3);
  } else {
    hue = randomised.hue - (i * 3);
  }
}
```

```
// Create our ellipse.
let ellipse = svg.create('ellipse');
ellipse.set({
  cx: center,
  cy: center,
  rx: radiusX,
  ry: radiusY,
  fill: 'none',
  stroke: `hsl(${hue} 80% 80% / 0.6)`,
  transform: `rotate(${rotation} ${center} ${center})`
});
}

/**
 * Gets a random number between a minimum and maximum value.
 */
function random(min, max, integer = true) {
  let random = Math.random() * (max - min) + min;
  let number = integer ? Math.floor(random) : random;
  return number;
}
```

Ok, so quite the code dump! It will no doubt overwhelm anyone new to coding, so if you fall into this category and find yourself balking at the aforementioned, please bear with me; all will be explained in the forthcoming chapters. The purpose of this first generative sketch is to just jump in and show you some quick results.

When you save the aforementioned, you should see something like Figure 1-3. Each refresh of the browser will render a unique version, so what you see will no doubt differ in some respects. But that, dear reader, is part of the joy of generative art.

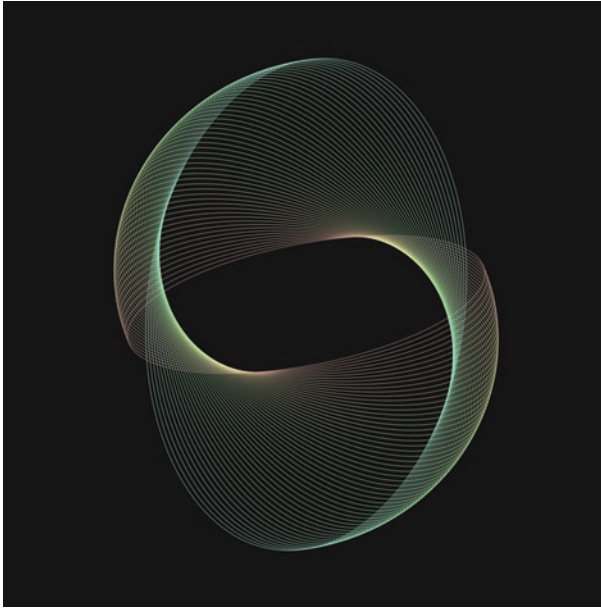


Figure 1-3. *Our first generative sketch (one variation of many)*

Summary

To recap, we've covered the following in this first chapter:

- Why we're using JavaScript and SvJs rather than Processing or p5.js
- What sets SVG apart from raster formats like PNG and JPG
- How scripting SVG differs from writing it directly in markup
- How SvJs can make SVG scripting more intuitive and less verbose

- Installing and importing the SvJs library and setting up our base template
- How to serve our sketches from a local development server

Coming up next: a programming primer.

CHAPTER 2

A Programming Primer

Although programming can be used to create art, it can also be an art in itself. It is part art and part science. When beginning programming, you need to familiarize yourself more with the scientific side of it, the fundamental concepts and rules that comprise programming as a discipline. The art can come later.

Learning the basics in programming means getting to grips with concepts like values, variables, operators, expressions, conditionals, loops, functions, and more. If you have no idea what any of these are, don't worry. That's what this chapter is for.

We'll also cover the characteristics (including some of the peculiarities) of JavaScript, our language of choice. JavaScript is a powerful and enormously popular language; it is the only programming language web browsers natively understand, so naturally enough, it is *everywhere*. Learning JavaScript is therefore a very practical choice and will serve you well in a lot of other areas besides generative art.

Syntax

We'll begin with some points on syntax.

Case Sensitivity

JavaScript is a case-sensitive language, so the words `JavaScript` and `javascript` would be considered distinct from one another. A capitalization convention that many coders adhere to when naming functions or variables is something called camel case, where terms are joined together in a manner resembling the humps of a camel. For example:

```
thisIsCamelCase  
soIsThis
```

As you can see, the first part of the term is entirely lowercase, and all subsequent terms use what we'd call title case, where the first character of the word is capitalized.

Spacing

Unlike some languages that enforce strict indentation, whitespace in JavaScript doesn't carry any intrinsic meaning. Spacing is mostly a matter of style. The most prevalent stylistic convention you'll come across within the JavaScript community is the use of two spaces to indent code, as follows:

```
someCode() {  
    some indented code  
}
```

Semicolons

Each statement in JavaScript should end with a semicolon (`;`), which is the equivalent of a full stop in natural language. You can choose to leave them out altogether, due to a JavaScript feature called Automatic

Semicolon Insertion (ASI), but because this isn't always safe to do (and because I also work with other languages where semicolons are mandatory), I opt to leave them in.

The code throughout this book will therefore use semicolons.

Comments

A very important habit to cultivate is the liberal use of comments in any code you write. Not only so that others can more easily read and understand your code, but so that *you* can do so too. You'd be amazed at how quickly your own code can become conundrum-like without comments to guide the way.

There are two types of comments in JavaScript: single-line comments and multi-line comments. Single-line comments begin with two forward slashes `//`, and multi-line comments begin with a forward slash and asterisk `/*` and terminate with an asterisk and forward slash `*/`.

```
// A single-line comment. Useful for quick explanations.  
  
/*  
 * A multi-line comment.  
 * Useful for properly documenting code.  
 */
```

Single-line comments can also be placed after code on the same line.

```
someCode(); // We can safely write a comment here.
```

Values

As a programmer, you've got to be well versed in values and their various types. We're not talking moral values here or a programming code of ethics; by values, we mean "chunks" of information that are eventually boiled down to the bytes and bits that the computer processes.

Values can be anything from numbers to strings of text (like "I love breakfast cereals"). Let's start with numbers.

Numbers

If computers love anything at all, it's numbers. They can crunch them far faster than I can crunch through my favorite breakfast cereal. In some programming languages, there are different types of numbers (e.g., `int` representing integers or whole numbers and `float` representing decimal or floating-point numbers), but in JavaScript, all numeric values are of the single type `Number`.

```
17 // A whole number.
```

```
23.085 // A decimal number.
```

```
4e2 // A number with an exponent (four to the power of two in  
this case).
```

Strings

Strings represent textual information, like words and sentences. Strings need to be surrounded by quote marks, of the single or double variety.

```
'There are 10 kinds of people in this world.' // Single quotes
```

```
"Those who understand binary and those who don't." //
```

```
Double quotes
```

Whether you choose to use single or double quotes, it's important to be consistent with your choice. Try not to mix them haphazardly.

Another type of string exists called the template literal, which allows you to insert variables and expressions into a string. This isn't possible with standard single or double quotes. Template literals are surrounded by backticks, and code can be inserted between curly braces prepended by a dollar sign, like so:

```
`Some text here ${someCodeHere}, and more text here. Pretty cool huh?`
```

This is a more recent feature of JavaScript and is extremely useful for handling concatenated (i.e., pieced together) and multi-line strings. Let's illustrate this with an example.

```
// Here's how developers used to have to store multi-line HTML string data.
```

```
let oldWay = '' +
'    <div class="intro">\n' +
'\n' +
'        <p>My name is ' + name + ' and I am ' + age + ' years
        of age.</p>\n'
'\n' +
'    </div>\n';
```

```
// And here's how the same thing can be done now.
```

```
let newWay = `
<div class="intro">

    <p>My name is ${name} and I am ${age} years of age.</p>

</div>`;
```

As you can see, it is much more readable and concise. As a rule, whenever you want to mix string values with anything else, backticks are the best choice.

Booleans

Boolean values are binary; they can be either true or false. They are named after George Boole, inventor of Boolean algebra.

They can be useful for representing any binary state: yes or no, on or off, alive or dead, etc. They are written as simply true or false.

```
let alive = true; // phew
let kicking = false; // just sitting
```

Booleans make conditional statements and comparisons possible, which we'll get to a little later.

Empty Values

Empty values are the final values we'll consider. There are two: null and undefined. You can think of them as values that carry no information. You'll see a lot of them (particularly undefined) when debugging your code, so as you can imagine, they are not always the most welcome of visitors.

What is the difference between null and undefined? A simplified way to think of it is this: when JavaScript tells you a variable called x is null, it's saying "yeah I know about x, but x doesn't have any value so far as I can see." If, on the other hand, it tells you that x is undefined, it is essentially saying "What the **** is x? Ain't no x around here."

Variables

We've name-dropped variables a couple of times already, so what exactly are they? You can think of variables like containers for values that can be referenced later.

A variable needs to be declared before it can be used. There are three ways to do this: using `const`, `let`, or `var`.

The latter, `var`, is no longer recommended; I mention it mainly for historical reasons and because it is something you'll likely encounter in the wild. Many, many JavaScript code bases out there still use `var`, simply because prior to the great JavaScript update of 2015 (called ECMAScript 6, or ES6 for short), there was no other options available.

Declaring a variable involves using the keyword, creating a name/identifier, and then assigning it a value using the equals operator.

```
const name = 'David Matthew';  
const countryOfBirth = 'Ireland';  
let age = 21; // I wish!
```

What's the difference between `const` and `let`? When using `const`, you're declaring a value that shouldn't change. My `countryOfBirth` is an example: this is a historical fact that remains constant. If I later tried to reassign a new value to `countryOfBirth`, this would result in an error:

```
countryOfBirth = 'Brazil'; // uh oh...  
-> TypeError: Assignment to constant variable
```

My age, however, is (alas) subject to change, so it can be updated without any issues. Another difference between `const` and `let` is that the latter can be declared without a value, whereas with `const`, a value must be assigned to it when it is first created.

```
let a; // A-ok.  
const b; // asking for trouble.  
-> Uncaught SyntaxError: Missing initializer in const  
declaration
```

When unsure, should you use `const` or `let`? The general consensus in the JavaScript community is that you should use `const` by default and only use `let` when you think the variable may be assigned a new value later. The reasoning is that this can reduce any unintended value re-assignments, ruling out a potential source of bugs.

Operators

An operator is a symbol that performs operations on values. We've actually come across one already: the assignment operator (`=`), which assigns a value to a variable. Let's see what other kinds of operators are available to us.

Arithmetic Operators

As you might have guessed, arithmetic operators allow us to perform mathematical operations. The addition operator (`+`) allows us to add numbers, but it also allows us to concatenate strings.

```
// Adding numbers.  
2 + 2  
-> 4  
  
// Concatenating strings.  
'I am' + ' so smrt'  
-> 'I am so smrt'
```

There are also operators for subtraction (-), multiplication (*), and division (/).

```
// Subtracting numbers.
```

```
9 - 4
```

```
-> 5
```

```
// Dividing numbers.
```

```
9 / 4
```

```
-> 2.25
```

```
// Multiplying numbers.
```

```
9 * 4
```

```
-> 36
```

ES6 introduced an exponentiation operator (**), which allows you to multiply one number by a factor of another (i.e., one number to the power of another).

```
// The exponentiation operator.
```

```
2 ** 2
```

```
-> 4
```

```
2 ** 6
```

```
-> 64
```

```
2 ** 10
```

```
-> 1024
```

And finally, there is the modulo operator (%). This divides one number by another and gives you the remainder. This can be especially useful when cycling through arrays (which we'll get to later), or quickly finding out if an unknown quantity is even or odd.

CHAPTER 2 A PROGRAMMING PRIMER

// The modulo operator in action.

$3 \% 3$

-> 0

$3 \% 2$

-> 1

$12 \% 5$

-> 2

At this point, it's worth mentioning rules of precedence, which determine the order in which operations are carried out. The mnemonic BEMDAS can help us out here.

- **Brackets:** Whatever is in brackets or parentheses is evaluated first.
- **Exponents:** Evaluate any exponents next.
- **Multiplication and Division:** Next, multipliers and divisors. These two have equal precedence, so they are evaluated in the order they are written, i.e., left to right.
- **Addition and Subtraction:** Add and subtract any remaining numbers. As with multiplication and division, these have equal precedence and are evaluated left to right.

// These sums look similar but result in different answers.

$4 / 2 * 2 + 4$

-> 8

$4 / 2 * (2 + 4)$

-> 12

$4 / 2 ** 2 + 4$

-> 5

When in doubt, use parentheses to group operations. This can also improve readability.

Comparison Operators

As the name suggests, comparison operators compare values. They always return a boolean, that is, a true or false value.

- `<` is the “less than” operator.
- `<=` is the “less than or equal to” operator.
- `>` is the “more than” operator.
- `>=` is the “more than or equal to” operator.
- `==` is the “equal to” operator.
- `===` is the “strictly equal to” operator.

Because of some subtle quirks with how JavaScript works, I would recommend sticking with the strict versions of the equality operators. This can reduce unexpected surprises (i.e., bugs).

```
// Some examples of comparison operators in action.
```

```
1 > 2
```

```
-> false
```

```
2 < 3
```

```
-> true
```

```
3 <= 4
```

```
-> true
```

```
5 == "5"
```

```
-> true // but not really.
```

```
5 === "5"
```

```
-> false // that's more like it.
```

Logical Operators

Lastly, there are the logical operators *and* (`&&`), *or* (`||`), and *not* (`!`). These emulate the conjunction, disjunction, and negation of classical logic.

When conjoining two conditions with `&&`, the result is true if and only if both conditions are true.

```
true && false
-> false
```

```
false && true
-> false
```

```
true && true
-> true
```

When using the disjunction (fancy word for *or*), first make sure you use the right characters on your keyboard. The `|` character is not a capital *i* or lowercase *L*; it's what's known as the pipe character. Depending on your regional keyboard layout, it might be located near either your Enter key or your Shift key.

When using `||`, the result is true if one or both conditions are true. This makes intuitive sense if we remember that we are essentially asking the computer “is at least one of these conditions true?”

```
true || false
-> true
```

```
false || true
-> true
```

```
false || false
-> false
```

Finally, we have the negation operator `!`. This “flips” the condition immediately following it. It can also be uniquely coupled with the equality operator to mean not equal to (`!==`).

```
!true
-> false

!false
-> true

1 !== 2
-> true

!(2 > 1)
-> false
```

A common source of error for new programmers is using a single pipe character `|` or a single ampersand `&` instead of the two together. By themselves, these characters perform bitwise operations (i.e., they manipulate binary values) and are for advanced programming only.

Conditional Statements

Now that we have comparative and logical operators under our belt, we can proceed to conditionals. A conditional statement in JavaScript allows us to influence a program’s control flow. As an analogy, think of rush hour traffic at a four-way intersection, the pavements packed with pedestrians. Without some way to control the flow of traffic, mayhem would ensue. Here is where conditionals are needed, such as the various rules that might underpin a traffic light system.

```
// The basic syntax.
if (conditionIsTrue) {
  // ... run some code
}
```

```
// A simplistic pedestrian crossing conditional.
if (pedestrianBtnIsPressed) {
    wait(30s);
    activateRedLights();
    wait(3s);
    activateGreenManSignal();
}
```

To the basic if statement, we can append else and else if conditions too.

```
if (a > b) {
    doThis();
} else if (a === b) {
    doThat();
} else {
    doOtherThing();
}
```

If a given condition isn't true, the code inside its curly braces doesn't get executed. Let's take the preceding example again, with the values filled in for a and b.

```
let a = 3;
let b = 4;

if (a > b) {
    doThis(); // condition wasn't true, so this code doesn't run.
} else if (a === b) {
    doThat(); // this code also doesn't run.
} else {
    doOtherThing(); // both the above are false, so this
code runs.
}
```

The last conditional form I want to cover is the ternary operator. As its name suggests, it is technically an operator, but because it plays the role of a conditional check, I felt it was more appropriate to include it here.

```
// The basic structure of the ternary operator.
conditionA ? expressionIfTrue : expressionIfFalse;
```

In the aforementioned, we’re asking “Is conditionA true? If so, run expressionIfTrue. Otherwise, run expressionIfFalse.”

The ternary operator is best used when you want to assign a value to a variable based on whether a certain condition holds true. If that condition doesn’t hold true, we then assign it a different value. This is exactly what I used it for in our first generative sketch:

```
const svgSize = window.innerWidth > window.innerHeight ?
window.innerHeight : window.innerWidth;
```

Here, we’re saying “If the window’s innerWidth is greater than its innerHeight, assign the innerHeight to svgSize. Otherwise, assign to it the innerWidth.”

Loops

Loops are one of a programmer’s superpowers. We’re talking sea-parting, earth-splitting powers. Or, at the very least, the ability to send your programs spiralling toward infinity, potentially freezing any open browser windows and spinning your CPU fan into a frenzy.

This is because loops can repeat blocks of code over and over, as many times as we want, or indefinitely if needed. However, it’s usually wise to set a limit to looping behavior (infinite loops are more often an error than an aim). We set a limit to a loop with what’s called a terminating condition, that is, a condition that tells the loop when to stop.

We'll cover two main kinds of loops here: the `while` loop and the `for` loop.

The While Loop

The `while` loop is the simpler of the two. It's basic structure is as follows:

```
while (condition === true) {  
  // run code here while condition is true.  
}
```

Now, if the condition in question *always* evaluates to `true`, that would lead to an infinite loop. An important ingredient therefore in any looping structure is a value that changes while the loop is running. This is normally a number that either increments or decrements and is known as an iterator. By convention, `i` tends to be used as the variable name.

```
// Set up the iterator.  
let i = 0;  
  
while (i <= 99) {  
  // Write the value of i to the browser console.  
  console.log(i);  
  // Increment i by 1 each time the loop runs.  
  i = i + 1;  
}
```

The preceding loop will run 100 times, starting at 0 and continuing until it is less than or equal to 99, and on each iteration, it will log the value of `i` to the browser console. If we left out the code to increment `i`, it would loop indefinitely.

It's worth noting at this point that there are shorthand ways of incrementing and decrementing – or indeed multiplying and dividing – variable values:

```
// Long way.
i = i + 1
i = i * 2
i = i - 1
i = i / 2

// Shorter way.
i += 1
i *= 2
i -= 1
i /= 2

// Even shorter way (only valid for adding or
subtracting by 1).
i++
i--
```

The shorthand notation is what you'll likely encounter in practice, so we'll continue to use it here.

The For Loop

The for loop uses the same logic as our example while loop but bakes the iterator and incrementation into a single line. Its typical structure is as follows:

```
for (let i = startingNumber; i < upperLimit; i += increment) {
    // run some code while i is less than the upperLimit.
}
```

The for loop structure is succinctly divided into three sections. We first initialize the iterator, then set the terminating condition, and then decide how we're going to vary the iterator. Here's the for loop version of our previous example, where we logged the value of `i` to the browser console on each iteration.

```
for (let i = 0; i <= 99; i++) {
  console.log(i);
}
```

You don't always have to increment the iterator. You can just as easily reverse the loop and have it count down from 99 to 0 by making a few tweaks.

```
for (let i = 99; i >= 0; i--) {
  console.log(i);
}
```

It is common to use the value of the iterator in the body of the for loop in combination with other variables to achieve some dynamic result. In our opening generative sketch for example, I used `i` to progressively increase the `radiusX` and `radiusY` of each ellipse.

```
// Run a loop (a randomised number of times) to create our
// ellipses.
for (let i = 0; i < randomised.iterations; i += 1) {
  // ... code.
  let radiusX = 100 + (i * 3);
  let radiusY = 300 + (i * 2);
  // ... more code.
}
```

There's lots more to love about loops, and other variations also exist (such as the `for in`, the `for of`, and the `forEach` which we'll cover later), but for the purposes of generative art, we'll be mainly utilizing the standard `for` loop.

Functions

Functions are fundamental to JavaScript as a language and are perhaps the single most important concept we'll cover in this chapter. Functions are self-contained blocks of code that perform a particular task. The task is defined in the function body, and it might range from adding two numbers together to initiating a rocket launch sequence.

Standard functions are defined using the `function` keyword followed by the name you give it, which is then appended with parentheses and opening and closing curly braces where the task is defined.

```
// Basic syntax of a standard function.
function performSomeTask() {
  // ... code defining the task.
}
```

Function Parameters

Although functions are used in a wide variety of ways, their primary purpose is to make code reusable. One way to do this is to add parameters to the function, which are defined within the parentheses of the function name.

```
// A function with parameters.
function sayHello(name) {
  // Greet the user the Irish way.
  console.log(`Howaya ${name}, what's the craic?`);
}
```

Functions may or may not explicitly return a value, and if they do, only one return value is allowed. In the preceding example, a value isn't returned, as that isn't the purpose of the function (or to get technical about

it, an undefined value would be returned if the function was queried). In the following example, however, we are specifically looking to get a value returned from the function, and this is where we use the return keyword.

```
// Another function with parameters, this time returning  
a value.  
function squareNumber(number) {  
    return number * number;  
}
```

The preceding function, when called, will return the result we're looking for, that is, the square of the number we supply it. But how do we actually do this in practice?

Invoking Functions

To use a function we've defined, we need to invoke it. This is synonymous with calling or executing the function. It can be done like so:

```
// Invoking a function.  
performSomeTask();
```

To invoke a function that has parameters as part of its definition, we supply it with arguments. Arguments are the actual values we give the function when we invoke it. The parameters, on the other hand, are how we define these values. Let's see how we might invoke a function with arguments.

```
sayHello('Dave');  
-> "Howaya Dave, what's the craic?"  
  
console.log(squareNumber(4));  
-> 16
```

Here, we've provided the argument 'Dave' for the first function and 4 for the second, and we've received the expected output. But what if we called the function and forgot the arguments? Let's try it out.

```
sayHello();
-> "Howaya undefined, what's the craic?"

squareNumber();
-> NaN
```

We've encountered undefined before, but not NaN. This just means "not a number." In both cases, our functions don't work so well without arguments. How might we fix this and make them a little more resilient?

One straightforward way is to assign default values when defining a function's parameters. Let's rewrite our sayHello() function as an example.

```
function sayHello(name = ", ah it's yourself") {
  // Greet the user the Irish way.
  console.log(`Howaya ${name}, what's the craic?`);
}

sayHello();
-> "Howaya, ah it's yourself, what's the craic?"
```

As you can see, creating a default value is as simple as assigning a value to the parameter when you define it. Now we've created a more human-sounding fallback than undefined.

In the case of the squareNumber() function, a default value doesn't make as much sense. Instead, we could use the ternary technique covered earlier to provide some actionable feedback if the function is called without an argument.

```
function squareNumber(number) {
  return number ? number * number : 'Please include the number
  you want to square';
}
```

In the `squareNumber()` function body, our ternary operator is essentially asking “Is `number` defined? If so, return the result of multiplying it by itself; otherwise, return our fallback message.”

At this point, you should be in a better position to fully understand the function defined in our first generative sketch. Have another look and see if it makes more sense now.

```
function random(min, max, integer = true) {
  let random = Math.random() * (max - min) + min;
  let number = integer ? Math.floor(random) : random;
  return number;
}
```

Scope

The scope of a variable refers to its visibility, or where in a program the variable can be accessed. It’s particularly important to be aware of when working with functions.

If, for example, we defined a variable within a function, it would only be visible within that function.

```
function scopeTest() {
  const a = 1;
}
```

```
console.log(a);
```

```
-> Uncaught ReferenceError: a is not defined
```